

Case 7: Filter by Query Parameter

Intermediate

Send a GET request with a query parameter and verify the filtered results.

New concept: Query parameters (params={})

API Endpoint

GET `https://jsonplaceholder.typicode.com/posts?userId=1`

What You Will Learn

- Query parameters — what they are and how URLs use them
- Passing `params={}` dict to `requests.get()`
- Validating that ALL items in a filtered list match the filter
- Using a list comprehension inside an assertion

Study Plan

Step 1 — What is a query parameter?

A query parameter is appended to the URL after a '?'. Example: `/posts?userId=1` filters posts to only those belonging to user 1. Multiple params use '&': `?userId=1&page=2`.

Step 2 — Two ways to pass params in requests

You can hardcode the URL: `requests.get('...?userId=1')`, or use the `params` argument (cleaner): `requests.get(url, params={'userId': 1})`. Both produce the same request.

Step 3 — Check the list is not empty

After filtering, `assert len(data) > 0` to ensure the filter returned results and didn't silently return nothing.

Step 4 — Validate every item matches the filter

Don't just check the first item. Use: `assert all(post['userId'] == 1 for post in data)`. This checks every single post in the list.

Step 5 — Check the count

JSONPlaceholder has exactly 10 posts per user. `Assert len(data) == 10` as an extra precision check.

Your Task

Write a pytest test that calls GET `/posts` with `userId=1` as a query parameter.

The test must verify:

1. Status code is 200
2. The response is a list with exactly 10 posts
3. Every post in the list has `userId == 1`
4. Every post has a non-empty title

BONUS: Parametrize the test to run for `userId 1, 2, and 3`

Hint: `requests.get(BASE_URL, params={'userId': 1})` — then use `all()` to check every item.

Base URL: <https://jsonplaceholder.typicode.com> | No authentication required | Level 2 — builds on Cases 1-5